

## MeteoCaracas.

Tu equipo de trabajo ha sido contratado para desarrollar un sistema de monitoreo y consulta del clima en el área Metropolitana de Caracas, en tiempo real. El cliente ha solicitado que el desarrollo se ejecute estrictamente bajo el enfoque de la **Programación Orientada a Objetos (POO)**, usando el lenguaje de programación Python.

A continuación, se describe el conjunto de requerimientos del sistema :

### Requerimientos funcionales:

1. Carga de datos: El sistema deberá leer el archivo **zonas\_caracas.json** al iniciar y transformar toda su estructura en una **lista de objetos** (ej. objetos de clase **Municipio** y **Localidad**). Tome en cuenta que muchas localidades no tienen coordenadas geográficas de manera que una vez terminada la carga del archivo en memoria, se deberá generar un reporte en pantalla indicando por cada municipio:
  - a. Cantidad de localidades cargadas.
  - b. Cantidad de localidades con coordenadas geográficas.
  - c. Cantidad de localidades sin coordenadas geográficas conocidas.
  - d. Porcentaje de localidades con coordenadas geográficas.
2. Consulta del clima en tiempo real: que es podrá hacer de dos formas:
  - a. Por municipio y localidad: El sistema desplegará la lista de municipios (Chacao, Baruta, El Hatillo, Sucre, Libertador). Al seleccionar un municipio, se mostrarán las localidades asociadas que posean coordenadas válidas. Luego, al seleccionar la localidad, el sistema consultará en tiempo real la API de Open-Meteo (<https://api.open-meteo.com/v1/forecast>) enviando la latitud y longitud correspondientes. Debe leer la documentación disponible de la API.
  - b. Mediante búsqueda directa por nombre de la localidad: El usuario podrá ingresar el nombre de una localidad (o parte de él). El sistema filtrará las coincidencias y le permitirá seleccionar la localidad deseada para obtener su clima. En ambos casos (puntos 2 y 3) el sistema deberá desplegar en pantalla los detalles meteorológicos de la localidad, a saber:
    - i. Nombre del Municipio y Localidad.
    - ii. Coordenadas (Latitud y Longitud).
    - iii. Temperatura actual (°C).
    - iv. Humedad relativa (%).
    - v. Velocidad del viento (km/h).

- vi. Código/Estado del tiempo (ej. Despejado, Nublado, Lluvia moderada).
- 3. Reportes y Estadísticas: El sistema deberá ofrecer un módulo de estadísticas que permita calcular e informar:
  - a. Ranking de Temperatura: Municipio con la localidad más cálida y más fría según las consultas realizadas en la sesión.
  - b. Cobertura Geográfica: Listado detallado de las localidades del archivo **zonas\_caracas.json** que no poseen coordenadas registradas (**null**), agrupadas por municipio.
  - c. Promedio General: Promedio de temperatura de las localidades consultadas durante la sesión activa.
- 4. Históricos:
  - a. Consulta por período de tiempo: El usuario podrá seleccionar una localidad con coordenadas válidas y especificar un rango de tiempo para el análisis. Deberá entonces indicar fecha de inicio y fecha de fin (tome en cuenta que la API utiliza el formato **AAAA-MM-DD**). El sistema deberá entonces mostrar por cada mes:
    - i. Temperatura (°C).
    - ii. Humedad relativa (%).
    - iii. Precipitación acumulada (mm).
    - iv. Velocidad del viento (km/h).
  - b. Deberá además, mostrar valores promedios de cada magnitud.
  - c. Indicar el año más caluroso y el más fresco, así como el año en el que hubo mayor precipitación y humedad relativa.
  - d. El sistema deberá mostrar un gráfico que permita comparar la evolución de cada magnitud en cada año dentro del período especificado.

## Requerimientos técnicos:

- I. **Almacenamiento de datos**: los datos se deberán almacenar en **listas de objetos**, por lo que se deberán diseñar las clases necesarias para almacenar la información.
- II. Deberá utilizar esta API para acceder a los datos meteorológicos: <https://open-meteo.com/>
- III. La lista de las localidades de cada municipio de la Ciudad de Caracas, se encuentra en el siguiente archivo: [https://drive.google.com/file/d/1Y9vEEGwJt-KpUmLzcj57UZ\\_s01sFDcU-/view?usp=sharing](https://drive.google.com/file/d/1Y9vEEGwJt-KpUmLzcj57UZ_s01sFDcU-/view?usp=sharing)

## Consideraciones

- El reto deberá ser entregado en **el aula virtual** a más tardar el **10 de agosto de 2026 a las 11:59 PM**.
- Los proyectos **podrán ser sometidos a defensa**, es decir, el facilitador convocará al equipo para una revisión.
- Los equipos de trabajo deberán utilizar [GitHub](#) para el control de versiones y facilitar el trabajo en equipo de manera remota. De esta forma, podrán comenzar a crear su

portafolio de trabajos, elemento que puede ser importante a la hora de buscar trabajo. Los registros de Github que tengan todos los commits de un solo integrante o en un solo día, serán considerados como **no válidos**, pues no reflejan la participación activa y significativa de todos los integrantes del equipo. Una participación se considera significativa si refleja que la adición o modificación de código, contribuye realmente al aumento del valor funcional de la solución. Prácticas deshonestas como el registro de commits en los que se simule la agregación de líneas de código, para luego eliminarlas en otro commits; el reiterado registro de commits en los que solo se adicionen comentarios, serán considerados como intento de fraude académico y en consecuencia, el repositorio no será válido. Los proyectos que no cuenten con un repositorio válido serán calificados con cero (0) puntos. Si existiese disparidad entre los aportes de los integrantes del equipo, el docente podrá asignar notas distintas a los integrantes del equipo, en función de los aportes realizados.

- Los proyectos que no estén debidamente documentados no serán aceptados y se les asignará **0 (cero)** puntos.
- Los proyectos que sean iguales o parecidos, serán calificados con **0 (cero)**.
- Los programas que “no corran”, serán calificados con **0 (cero)**.
- Los proyectos que sean generados por herramientas derivadas de ChatGPT, serán calificados con **0 (cero)**.
- Solo se permite el uso de las librerías **requests, numpy, pandas, Matplotlib**.
- La información obtenida de las APIs estará conformada por listas y diccionarios, esta información se debe transformar y guardarse utilizando **listas y objetos**, está prohibido el uso de diccionarios para guardar la información de las APIs. En caso de no utilizar **Programación Orientada a Objetos (POO)** los proyectos serán evaluados con **0 (cero) puntos**.
- Todas las funciones y métodos del proyecto deben estar documentadas usando **Docstring**.
- Los equipos pueden tener como **máximo 3 personas**.

## Criterios de evaluación

- **Funcionalidad:** Capacidad para proporcionar las funcionalidades que satisfacen las necesidades explícitas e implícitas bajo unas ciertas condiciones. **(65%)**
  - **Adecuación:** El programa ofrece todas funcionalidades que respondan a las necesidades, tanto explícitas (contenidas en el documento descriptivo del proyecto) como implícitas; entendiendo como necesidades implícitas, aquellas que, no estando descritas en el documento, surgen como resultado de un concienzudo análisis del problema planteado y que aseguran el correcto funcionamiento del programa.

- *Exactitud:* El programa genera los resultados o efectos correctos o acordados, con el grado necesario de precisión.
- *Fiabilidad:* Capacidad para mantener un nivel especificado de prestaciones cuando se usa bajo ciertas condiciones.
  - *Madurez:* El programa no presenta fallas originadas por errores de programación, análisis o diseño. **(10%)**
  - *Tolerancia a fallos:* El programa responde adecuadamente al manejo inadecuado del usuario; es decir, mantiene su correcto funcionamiento aun cuando el usuario introduzca datos erróneos o manipule inadecuadamente las interfaces de usuario. **(10%)**
- *Usabilidad:* Capacidad del proyecto para ser entendido, aprendido, usado y al mismo tiempo, ser atractivo para el usuario, cuando se usa bajo condiciones específicas.
  - *Comprensibilidad:* El programa ofrece una interfaz de fácil comprensión, facilitando su aprendizaje y correcta utilización. El programa emite mensajes de alerta cuando se introducen valores erróneos. Existen elementos informativos que indican al usuario como operar el programa. **(5%)**
- *Eficiencia:* Capacidad para proporcionar prestaciones apropiadas, relativas a la cantidad de recursos utilizados, bajo condiciones determinadas.
  - *Estructuras de datos:* Utiliza eficientemente los principios y buenas prácticas asociadas a la POO. **(10%)**